01 — CHAPTER NOTES (full, in slide order)

Everything below comes straight from the slide decks, kept in lecture order. Short blocks; examples sit on their own lines.

> **Which file is which chapter** (names are shifted +1): `Ch.02` = Ch 1 · `Ch.03` = Ch 2 · `Ch.04` = Ch 3 · `Ch.05` = Ch 4 · `Ch.06 Software Testing` = Ch 5 · `Ch.06 Unit Testing` = Ch 6.

CHAPTER 1 — Basics of Software Quality Assurance

What is software?

Software is **more than a computer program** — it is programs, procedures, and possibly associated documentation and data. Two major types: - **Generic (Buy)** — stand-alone, sold on the open market → **COTS** (commercial off-the-shelf). - **Customized (Build)** — built for a specific customer/business (bespoke). *Example:* a COTS office suite is used by thousands and may not do exactly what you want; bespoke software is built to one business's exact requirements.

What is software quality?

- **ISO/IEC 9126:** the totality of functionality and features that contribute to a product's ability to satisfy **stated or implied needs**.
- **IEEE Std 610:** the degree to which a component/system/process meets specified requirements and/or user needs and expectations.

Challenges in software projects

The primary goal of project management: deliver **on time, on budget, at expected quality**. Three main challenges: **Time** (late), **Cost** (over budget), **Scope** (low quality/faults).

Examples of software defects

- **Therac-25** — radiation therapy machine by AECL (1986); people died from massive radiation overdose caused by a **software bug**.
- **Ariane 5** — 4 June 1996; the rocket tore itself apart **37 seconds** after launch due to a control-software malfunction — the **most expensive computer bug in history**.

Software testing

Testing = executing a system/component under specified conditions to **find defects/bugs** and verify it satisfies requirements. Goals — detect issues in: - Requirement conformance - Errors in system operation - System performance *(example: TCAS in an autopilot system)* Levels: **Unit** (each module), **Integration** (combine modules, check incompatibility), **System** (software + hardware combined).

Role of testing — two categories

- **Static analysis (manual testing):** examines the code/document and reasons over **all** behaviours that might arise at run time. *Examples:* code review, inspection, algorithm analysis. *Limitation:* review takes a long time.
- **Dynamic analysis (automated testing):** actual **program execution** to expose failures; observe representative behaviour, then conclude about quality. *Limitation:* cannot "think out of the box"; finds only syntax errors. Static and dynamic are **complementary** — combine the strengths of both.

Software Quality Assurance (SQA)

The function that **assures** standards, processes and procedures are appropriate and correctly implemented — a **planned and systematic** approach. - An **umbrella activity** applied throughout the software process. - Monitors

the engineering processes/methods to ensure quality. - *Role:* develop an effective **plan** for software development.

Software Quality Control (SQC)

The function that **checks** the project follows its standards/processes/procedures and produces the required internal & external (deliverable) products. - *Role:* **execute** the plan effectively. - Activities: **Requirement Review, Design Review, Code Review, Deployment Plan Review, Test Plan Review, Test Cases Review**.

Difference between SQA & SQC

- SQA gives **assurance** quality will be achieved; SQC **fulfils** the quality request.
- SQA aims to **prevent** defects; SQC aims to **identify and resolve** defects.
- SQA is the technique of **managing** quality; SQC is the method to **verify** quality.
- SQA **does not execute** the program; SQC **always executes** the program.
- SQA = **all team members** responsible; SQC = **only the testing team**.
- SQA = **Verification** (doing things right, process-oriented); SQC = **Validation** (results as expected, product-oriented).
- SQA = planning; SQC = action/execution.
- SQA = full **SDLC**; SQC = software **testing life cycle**.
- SQA = **failure prevention** system; SQC = **failure detection** system.

Difficulties in achieving good quality

- **Size** — many LOC; manual testing struggles to find logical errors in bulky code.
- **Complexity** — software deals with logic; a logic error can have big impact *(example: autopilot "set landing mode")*.
- **Environmental stress/constraints** — test at maximum load *(example: result-publish website)*.
- **Flexibility/adaptability** — frequent change *(example: regression testing after each change)*.
- **Cost and market conditions** — testing cost of automated tools and licenses.

Software Quality Engineering (SqE)

Ensures quality through **validation and verification** activities. Three stages: - **Planning** — quality planning. - **Execution** — execute the chosen QA plan / V&V activities. - **Decision** — decide pass/fail criteria of a test case; measurement & analysis to demonstrate quality.

SqE activities

- **Testing** — remove defects and ensure quality.
- Other QA alternatives to testing:
- **Defect Prevention** (plan to avoid defects)
- **Formal Verification** (Inspection / Review / Walkthrough)
- **Fault Tolerance** *(examples: data backup, auto-save document, reliability up to 5 failures)*

Error, Fault, Failure, Defect

- **Error** — a human action that produces an incorrect result (missing/incorrect human action that injects a fault).
- **Fault** — an incorrect step, process, or data definition in a program (underlying condition that causes failures).
- **Failure** — the inability of a system/component to perform its required function within specified performance requirements.
- **Defect** — a behavioural deviation from the user requirement or product specification. Failures, faults, and errors are collectively called **defects** — also commonly called **"bugs"**.

Complete testing

Objectives: detect bugs, reduce risk of defects, reduce cost of testing. **Complete/Exhaustive testing** = no undisclosed faults at end of test phase — **nearly impossible** because: - the input domain (valid + invalid) is too large; - the design may be too complex, and budget too high; - you cannot create all possible execution environments *(example: every autopilot simulation)*.

Testing activities

Identify the objective → select inputs from the input domain → compute expected outcome → set up execution environment → execute → analyse results vs expected. A test exercises a **subset** of the input domain and a subset of program behaviour (divide domain D into D1, D2).

CHAPTER 2 — Software Quality

Quality perspectives

- **Subjects (people's perspectives): External/Consumer** (customers, users) · **Internal/Producer** (developers, testers, managers) · **Other** (3rd party, indirect users, e.g. email notification).
- **Objects:** software products, systems, and services.
- **External view** = mostly a **black box** (observe behaviour, can't see inside).
- **Internal view** = a **white/clear box** (see what is inside and how it works).

Five views of quality (Kitchenham & Pfleeger)

1. **Mystical view (misconception)** — quality is recognized through experience but not definable; a good-quality object "stands out" and is easily recognized.
2. **User view** — quality = extent a product meets user needs/expectations; good if it satisfies many users; covers usability, reliability, efficiency.
3. **Manufacturing view** — **conformance** to process standards/requirements; deviation reduces quality; "**right the first time**" to cut cost; improve product by improving process. **CMM and ISO 9001** are based on this view.
4. **Product view** — good **internal** properties lead to good **external** properties.
5. **Value-based view** — customers' **willingness to pay**; merges **excellence** (a measure of quality) and **worth** (value); trades off cost vs quality.

Why measure quality?

Measurement gives a **quantitative** view. Reasons: - **Baseline** — establish quality baselines *(example: extract all info from a website within 20 minutes)*. - **Quality improvement based on cost** — each improvement has a cost; measurement drives process improvement. - **Know the present level** — for future planning; investigate improvement needs after measuring.

Software Quality Factor

A quality **factor** = a **behavioural characteristic** of a system. *Examples:* Correctness, Reliability, Efficiency, Performance. Model used: **McCall's** Quality Factors and Criteria. - **Integrity/Security** — extent unauthorized access to software/data can be controlled *(example: only Auditor-privileged users can view transaction histories)*. - **Efficiency** — computing resources and code required to perform a function *(example: ≥25% of processor/RAM unused at peak load)*. - **Usability** — effort to learn, operate, prepare input, interpret output *(example: submit a request in avg 4 / max 6 minutes)*. - **Reliability & Correctness** — extent a program performs intended functions with required precision; **reliability = probability of executing without failure for a specific period of time** *(example: ≤5 of 1000 runs lost to failures)*.

Software Quality Criteria

A quality **criterion** = an **attribute of a quality factor** related to software development. *Examples:* Modularity, Testability, Maintainability, Reusability. - **Modularity** — architecture attribute; cohesive components in one module → higher maintainability. - **Maintainability** — effort to locate and fix a defect in an operational program. - **Testability** — effort to ensure a program performs its intended functions *(example: max cyclomatic complexity of a module ≤ 20)*. - **Flexibility** — effort to modify an operational program. - **Portability** — effort to transfer a program from one hardware/software environment to another. - **Reusability** — extent parts of a system can be reused in other applications. - **Interoperability** — effort to couple one system with another *(examples: biometric SIM registration, blockchain, importing chemical structures from another tool)*.

ISO-9126 Quality Framework

- The **most influential** framework in software engineering today.
- A **hierarchical** framework: **quality characteristics** and **sub-characteristics**.
- **Six top-level characteristics**, each with its own **non-overlapping (exclusive) sub-characteristics**: 1. **Functionality** — what is needed 2. **Reliability** — functions correctly 3. **Usability** — effort to use 4. **Efficiency** — resources needed 5. **Maintainability** — correct/improve/adapt 6. **Portability** — from one environment to another

Quality expectations

- **External/Consumer** — "good enough" for the **PRICE**: **Fit-for-use** (doing the right things) + **Conformance** (doing the things right). Expectations differ: General (functionality & reliability), Usability (GUI/web), Safety (safety-critical systems, e.g. autopilot).
- **Internal/Producer** — "good enough" for the **COST**: mirrors the consumer side; functionality & correctness via V&V; maintainability (service); interoperability (interfacing units); modularity (3rd party/outsourcing).

CHAPTER 3 — Maturity Models

Software process

A **process** = a set of activities executed to develop products (methods, techniques, strategies, procedures, practices) relying on repositories (documents, standards, policies). Benefits of a defined process: it can be **repeated**, **evaluated** (via metrics like cost, quality, time), and **improved**. Process tasks: Requirements Analysis, Design/Modeling, Coding, Testing, Implementation/Integration, Operation/Maintenance, Documentation.

Process improvement (Verification)

To improve a defined process (from a **baseline** = existing practices), evaluate capabilities and limitations. Why improve a dev/test process: - **Quality** — better insight into quality characteristics. - **Lead Time** — saves testing time. - **Cost** — lower cost. Three models: - **CMM** — evaluate software **development** processes; supports incremental improvement. - **TPI** — improve the **testing** process. - **TMM** — evaluate a **testing** process.

Capability Maturity Model (CMM)

An organization's **maturity level** tells to what extent it can produce **low-cost, high-quality** software. Knowing the current level, it can target the next. **Five levels:** 1. **Initial** — processes disorganized/chaotic; success depends on individual efforts; not repeatable (not sufficiently defined/documented). 2. **Repeatable** — basic project-management techniques established; successes can be repeated. 3. **Defined** — organization has its own standard process (documentation, standardization, integration). 4. **Managed** — organization monitors and controls its processes through **data collection and analysis**. 5. **Optimizing** — processes constantly improved through **feedback** and **innovative** processes.

Test Process Improvement (TPI)

A test process = a way of performing activities related to **defect detection**, e.g. identifying test goals, preparing a test plan, identifying kinds of tests, hiring test personnel, designing test cases, procuring tools, assigning cases to engineers, prioritizing, organizing execution into cycles, executing, reporting defects. **How to improve a test process (4 steps):** 1. Determine an area for improvement. 2. Evaluate the current state (baseline — quality, time, cost). 3. Identify the next desired state and the means to achieve it. 4. Implement the necessary changes.

Testing Maturity Model (TMM)

Like development-process improvement, testing processes also need a framework to assess and improve; evaluation is key. - **Pioneered by Ilene Burnstein**. - Describes an evolutionary path of test-process maturity in **five levels**. - Each level = **Maturity goals**, **Supporting maturity goals**, and **ATRs** (Activities, Tasks, Responsibilities — views from manager, developer, tester, customer).

TMM levels: 1. **Initial** — no maturity goals; testing begins **after** code is written; done to show the system works; ad-hoc; progress not tracked; testing not seen as critical. 2. **Phase Definition** — develop testing & debugging goals; initiate a **test planning** process (identify objectives, analyze risks, devise strategies, develop specs, allocate resources); institutionalize basic testing techniques. 3. **Integration** — establish a **software test group**; establish a technical training program; integrate testing into the software lifecycle; control and monitor the testing process. 4. **Management and Measurement** — establish an org-wide review program; establish a test management program; evaluate software quality. 5. **Optimization / Defect Prevention and Quality Control** — apply process data for **defect prevention**; **statistical quality control**; test process **optimization**.

CHAPTER 4 — Software Quality Assurance

Quality Assurance

QA focuses on the **correctness** aspect of quality and dealing with defects. Three generic categories: - **Defect Prevention** — prevent certain faults from being **injected** (errors = missing/incorrect human actions) *(examples: default values, options to select)*. - **Defect Reduction** — detect and remove faults once they've been injected. - **Defect Containment** — control defects via fault tolerance, failure prevention, or failure-impact minimization to assure reliability & safety *(example: auto-save)*.

Defect prevention

Two generic ways: - **Error source removal** — eliminate error sources (ambiguities, human misconceptions) via **education and training** (people-based solution). - **Error blocking** — directly correct/block the missing or incorrect human actions *(examples: user can't set the divisor to 0; data validation in Excel)* — a direct intervention that prevents fault injection. Also: **systematic defect prevention via process improvements** — formal methods prevent deviations from specs/design.

Defect reduction

Prevention is **not 100% effective**, so reduce faults with: - **Inspection** — the most common **static** technique: critical reading/analysis of code, designs, specs, test plans. - **Informal reviews/walkthroughs** — self-conducted, independent, pass-around, "canteen discussion". - **Formal inspections** — multiple human inspectors, predefined coordination. - **Testing** — execute the software, observe behaviour; on failure, analyze to locate & fix the fault. - **Other / risk identification** — formal model-based analyses, **boundary value analysis**, **control-flow & data-flow analyses**, **simulation & prototyping** *(example: autopilot)*.

Defect containment

Reduction only lowers faults to a low level, not zero (large size, high complexity, complete testing impossible). Containment focuses on **failures** — contain them to local areas (no global failure) or limit damage *(example:*

exception handling). Two generic ways: - **Software fault-tolerance** — **Recovery** (rollback and redo) · **NVP** (N-Version Programming — fault blocked). - **Safety assurance and failure containment.**

Safety assurance & failure containment

- **Safety** — accident-free *(example: autopilot safety eliminates pilot error)*.
- **Accident** — a failure with severe consequences.
- **Hazard** — a pre-condition to an accident.
- **Safety assurance** — hazard analysis, hazard elimination/reduction/control, damage control.

QA in the software process

- **Mega-process:** Initiation → Development → Maintenance → Termination.
- **Development components:** Requirement, Specification, Design, Coding, Testing, Release.
- **Process variations:** Waterfall · Iterative (QA in increments) · Spiral (QA + risk management) · Agile (XP — test-driven development, pair programming) · Maintenance (focus on defect handling).

Mapping DC view → V&V view

Defect-Centered (DC) activity → Verification/Validation focus: - **Defect prevention** → Both, mostly indirectly (Requirement-related = Validation; Project Plan = Verification; Formal specification = Validation). - **Defect reduction (testing)** → Both, mostly **Verification**. Unit = **Verification**; Integration = both (more verification); System = both; Acceptance = both (more validation); **Beta = Validation**. - **Defect containment** → Both, mostly **Validation**. Operation = Validation; Design & implementation = both (more verification).

CHAPTER 5 — Software Quality Engineering (SQE)

What SQE is

Meet or exceed quality expectations through the selection and execution of appropriate QA activities while **minimizing cost and project risks** under project constraints. It is an integral part of the overall software engineering process (cost and schedule are also managed).

Generic testing process

- **Pre-QA activities — Quality/Test Planning:** most key decisions are made here. Set specific quality goals:
- Identify quality perspective & expectation (meaningful to target customers/users).
- Select **direct quality measures** (quantified values for efficiency, reliability, usefulness).
- Assess quality expectations vs **cost**. Form an overall QA strategy (low-level, test preparation): select appropriate QA activities; choose quality measurements/models.
- **Test procedure preparation — Test cases (micro-level):** a **test case** is a collection of entities and related information that allows a test to be executed; includes test-case allocation and sequencing from **simple to complex**.

Test plan vs Test case vs Test suite

- **Test plan** — a **high-level** document describing objectives, scope, approach, resources, schedule, and focus of testing activities.
- **Test case** — a **low-level** document describing an input/action/event and its expected results, to determine if a feature works correctly.
- **Test suite (macro-level)** — the collection of individual test cases run in a sequence until stopping criteria are met.
- Reuse of test cases from earlier versions = **regression testing**.

- All test cases should form one integrated suite regardless of origin. *(A test-case template includes: Test Case ID, Priority, Module, Title, Description, Precondition, Dependencies, Test Steps, Test Data, Expected Results, Actual Results, Status Pass/Fail, Post Condition.)*

Test execution & post-QA

- **In-QA — Test Execution:** execute planned QA activities and handle defects; collect failure info (what/where/when/severity); document activities (templates).
- **Post-QA — Quality Measurement, Assessment & Improvement:** follow-up, feedback, identify improvement potential. **"Post-QA" ≠ after QA finishes** — measurement/analysis runs **parallel** to QA; pre-QA may overlap QA too.

Testing teams — organization & management

- Customers/users may act as **informal** testers (usability/beta).
- Independent professional testing organizations act as a **trusted intermediary** between vendors and customers.
- Team structures:
- **Vertical model** — organized **around a product**; dedicated people perform one or more testing tasks for that product.
- **Horizontal model** — performs **one kind of testing** for many different products.
- **Mixed model** — combines both; used by large software organizations.

CHAPTER 6 — Software Testing

Testing basics

Testing = execute the software and observe its behaviour/outcome. On failure, analyze the record to locate and fix the fault; otherwise gain confidence the software will do its job. Testing is a **distinct process**: reveals defects, shows quality attributes (reliability, performance); begins when the project is conceptualized; done by different people at different stages; uses many strategies/metrics; shaped by organizational policy; combines **manual and automated** execution.

Seven principles of testing

1. **Testing shows the presence of defects** but cannot prove their absence.
2. **Exhaustive testing is impossible** — use risk analysis, time/cost analysis, and priorities.
3. **Early testing** — defects found earlier are easier and cheaper to fix.
4. **Defect clustering** — a few modules contain most defects; defects are **not** evenly distributed.
5. **Pesticide paradox** — repeating the same tests stops finding new defects; review and revise tests.
6. **Testing is context dependent** — safety-critical software is tested differently from an e-commerce site.
7. **Absence-of-errors fallacy** — finding/fixing defects doesn't help if the system is unusable or doesn't meet user needs; no defects found ≠ ready to ship.

Testing levels (and who does each)

- **Unit testing** — *Developer*: test individual units (procedures, methods) in isolation.
- **Integration testing** — *Tester*: assemble modules into larger subsystems and test.
- **System testing** — *Tester*: test the whole system (functionality, load, etc.).
- **Acceptance testing** — *End-user*: verify the customer's expectations.
- **Regression testing** — **no new test cases designed**; existing tests are selected, prioritized, and executed to ensure nothing broke in the new version.

Testing & debugging (roles)

- Test and re-test = **test** activities; debugging & correcting defects = **developer** activities.
- **Developer** — implements requirements, designs & programs; creating a product is success; perceptions **constructive**; software is known and driven by **delivery**.
- **Tester** — plans testing, designs test cases; only concerned with finding defects; finding errors is success; perceptions **destructive**; software is unknown and driven by **quality**.

Testing key questions

- **WHY** — demonstrate proper behaviour/quality; defect-free development (detection & removal).
- **HOW** — techniques/process: **test planning & preparation → test execution → analysis & follow-up**.
- **VIEW** — Functional/external/**black-box**; Structural/internal/**white-box**; **Gray-box** (mixed).
- **EXIT** — Functional coverage vs usage-based (quality/reliability goals).

Testing techniques

- **White-box (WBT)** — implementation-based / **structural**; examines source code focusing on **control flow** and **data flow**; done by developers. Derives test cases that: exercise all independent paths at least once; exercise all logical decisions true & false; execute all loops at their boundaries; exercise internal data structures.
- **Black-box (BBT)** — specification-based / **functional**; examines the program from outside at the external interface; apply input, observe outcome. The SQA group finds: incorrect/missing functions; interface errors; external database access errors; behaviour/performance errors; initialization/termination errors.
- **Levels of abstraction:** high-level whole system → **black-box** (late); low-level statements/data → **white-box** (early/small); middle levels → **gray-box** *(example: procedures individually as black box, procedure interconnection as white box at module level)*.

WBT vs BBT (detailed)

- **Perspective:** BBT = black box (input–output/external behaviour); WBT = glass box (internal implementation visible).
- **Objects:** WBT for small objects; BBT for large systems.
- **Timeline:** WBT early (unit/component); BBT late (system/acceptance).
- **Defect focus:** BBT observes external-function failures; WBT observes internal-implementation failures.
- **Detection & fixing:** WBT defects are easier to fix; WBT may **miss omission/design problems** (BBT catches these); BBT is effective for interface/interaction problems, WBT for problems localized in a small unit.
- **Techniques:** BBT if external functions are modeled; WBT if internal implementations are modeled.
- **Tester:** BBT by professional testers / third-party (IV&V); WBT by developers.

When to stop testing (exit criteria)

- "Not finding defects anymore" is **NOT** an appropriate stopping criterion — user acceptance is what matters at the end.
- **Resource-based** — stop when time/money runs out (irresponsible for product quality).
- **Quality-based** — stop when quality goals (usability, reliability, real customer-usage resemblance) are reached.
- Substitute: **activity completion** — stop when planned test activities are done.

Manual vs Automated testing

- **Manual** — oldest and most rigorous; a tester performs operations; hard to repeat, not always reliable, costly, time-consuming, labour-intensive.
- **Automated** — uses software tools; runs without manual intervention; fast, repeatable, reliable, reusable, programmable, saves time.

Test automation

- **Key issues:** needs/potential for automation & tool selection; user training & time/effort; overall cost (acquisition, support, training, usage); impact on resources/schedule/management.
- **Pre-requisites:** system stable & well-defined; test cases unambiguous; tools & infrastructure in place; engineers have prior automation experience; adequate budget.
- **Automate:** tests run for every build (repetitive/**regression**); data-driven tests (multiple values); tests needing internal details (GUI attributes); **stress/load** testing.
- **Don't automate:** usability testing; logical errors; specification/design documents; one-time / "ASAP" testing; ad-hoc/random testing; tests without predictable results.
- **Automated testing CANNOT replace manual testing.**